"Año de la Esperanza y el Fortalecimiento de la Democracia"

Universidad Nacional de Ingeniería
Facultad de Ciencias
Escuela Profesional de Ciencia de la Computación

Desarrollo de Software - CC3S2A

Laboratorio 1 semana 2:

Integrantes	Código
Inga Chavez, Jorly	20220467J

Docente: Quispe Torres Manuel Alejandro

2026

Índice

1. Pregunta 1. **3**
1.1. Introducción al Problema 3
1.2. Diseño de Funciones y Funcionalidad General 3
1.3. Estructuras de Datos Utilizadas 3
1.4. Lógica de Control y Algoritmo de Búsqueda 4
1.5. Salida del programa 4

2. Pregunta 2. **5**
2.1. Introducción al Problema 5
2.2. Diseño de Funciones y Funcionalidad General 5
2.3. Estructuras de Datos Utilizadas 5
2.4. Lógica de Control y Algoritmo de Evaluación 6
2.5. Salida del programa 6

1. Pregunta 1.

1.1. Introducción al Problema

El objetivo del programa es desarrollar de una solución algorítmica diseñada para crear una función que simule el comportamiento de enrutamiento en una aplicación de una sola página.

Para cumplir con los requerimientos, el algoritmo debe manejar rutas estáticas y dinámicas, considerando que algunas rutas pueden incluir parámetros. Estos parámetros se identifican mediante el uso de dos puntos al inicio del segmento, como en el caso de */user/:id*. Si el sistema determina que una transición no coincide con ninguna ruta registrada, debe manejar la excepción devolviendo el mensaje "404 Not Found".

1.2. Diseño de Funciones y Funcionalidad General

La arquitectura del código se diseñó priorizando la modularidad y las buenas prácticas de programación en Python:

- **Encapsulamiento:** La función `resolver_enrutamiento()` agrupa toda la lógica de lectura, procesamiento y salida de datos. Esto evita la contaminación del espacio de nombres global y facilita el aislamiento del entorno de ejecución, lo cual resulta ideal para realizar pruebas unitarias.
- **Punto de Entrada Seguro:** Se implementó la condición `if __name__ == "__main__":` al final del script. Esta estructura garantiza que la función de enrutamiento solo se ejecute cuando el archivo es invocado directamente desde la consola, permitiendo que el código sea importado como un módulo en otros proyectos sin disparar su ejecución de forma accidental.

1.3. Estructuras de Datos Utilizadas

- **Listas (`list`):** Se utilizó una lista denominada `rutas` para almacenar las configuraciones del sistema.
- **Tuplas (`tuple`):** Cada ruta ingresada se almacena en la lista principal como una tupla con dos elementos: la ruta fragmentada y su contenido principal.
- **Cadenas de texto (`str`) y sus métodos:**
  - El método `strip()` se aplica a cada línea de entrada para eliminar espacios en blanco o saltos de línea residuales, asegurando una lectura limpia.
  - El método `split('/')` constituye la piedra angular del análisis estructural. En lugar de comparar las URL como textos completos, este método las divide en segmentos independientes, lo que permite evaluar jerarquías y detectar parámetros de forma aislada.

1.4. Lógica de Control y Algoritmo de Búsqueda

El núcleo del sistema radica en cómo procesa las entradas (donde la primera línea determina el número de rutas `N` y posteriormente el número de transiciones `M`) y cómo cruza esta información. Se aplican los siguientes mecanismos:

- **Filtrado por Longitud:** Para optimizar recursos, el primer filtro que aplica el sistema al comparar una transición con una ruta es verificar la cantidad de segmentos (`len(partes_ruta) == len(partes_trans)`).
- **Iteración Paralela (`zip`):** Se utiliza la función integrada `zip()` para comparar en paralelo los segmentos de la ruta predefinida y los de la transición actual.
- **Detección de Parámetros Dinámicos:** Durante la iteración paralela, el algoritmo utiliza el método `startswith(':')` sobre el segmento de la ruta. Si resulta verdadero, el sistema comprende que no debe buscar una coincidencia exacta de texto, sino que debe capturar el valor proporcionado en la transición para añadirlo después del contenido, seguido de un espacio.
- **Banderas de Control:** Se implementaron las variables booleanas `coincide` y `encontrado`. La bandera `coincide` verifica la viabilidad de una sola ruta, mientras que `encontrado` monitorea el estado global de la búsqueda. Si tras evaluar todas las configuraciones disponibles `encontrado` se mantiene como falso, se activa el mecanismo de manejo de errores, imprimiendo el mensaje `"404 Not Found"`.

1.5. Salida del programa

```
... 3
    / HomePage
    /profile ProfilePage
    /user/:id UserPage {id}
    4
    /
    HomePage
    profile
    404 Not Found
    /user/42
    UserPage 42
    /settings
    404 Not Found
```

Figura 1: Salida del programa

2. Pregunta 2.

2.1. Introducción al Problema

Objetivo del Programa: Explicar que el propósito del algoritmo es procesar un registro de transacciones bancarias para identificar al cliente más fiel de cada socio del Banco de la Nación.

2.2. Diseño de Funciones y Funcionalidad General

- **Encapsulamiento:** La función `resolver_fidelidad_clientes()` agrupa toda la lógica de lectura, asociación de variables y evaluación de datos.
- **Procesamiento de Entrada:** Se implementó la condición `if __name__ == "__main__":` al final del script. Esta estructura garantiza que la función principal solo se ejecute cuando el archivo es invocado directamente desde la consola

2.3. Estructuras de Datos Utilizadas

- **Listas (list):** Se utilizaron para almacenar y manipular transitoriamente los datos de entrada, por ejemplo cuando usamos el metodo `split()` (ya sea sobre `sys.stdin.read()` o `input()`)
- **Tuplas (tuple):** Son importantes en el algoritmo de desempate dentro de la función `max()`. Al agrupar los criterios en una tupla de la forma `(clientes_del_socio[x], -x)`, el sistema evalúa jerárquicamente
- **Cadenas de texto (str):** Estan presentes en las fronteras del programa. Se utilizaron al inicio para recibir la captura de datos planos (antes de convertirlos numéricamente con `int()`) y en la etapa final mediante cadenas formateadas o `f-strings` (por ejemplo: `f"{socio} {cliente_mas_fiel}"`), asegurando que la consola imprima los espacios exactos.
- **Tablas Hash (Diccionarios):**
  - **Diccionario de Mapeo de Terminales:** Se estableció la estructura `id_terminal: id_socio`. Al aprovechar la regla de que los terminales son únicos para cada socio, esta estructura actúa como un índice rápido.
  - **Diccionario Anidado para Agrupación de Conteos:** Se diseñó una estructura multidimensional de la forma `id_socio: id_cliente: cantidad_de_compras`. Este diseño organiza jerárquicamente las operaciones: agrupa las interacciones en el espacio del socio y mantiene un acumulador individual de las transacciones por cada cliente.

2.4. Lógica de Control y Algoritmo de Evaluación

El núcleo analítico del programa fluye a través de una fase de acumulación de datos y una fase final de evaluación estructurada. Se aplican los siguientes mecanismos:

- **Mapeo Directo (Búsqueda en Tabla Hash):** Para optimizar recursos y evitar bucles anidados ineficientes, el programa identifica a qué socio pertenece una compra buscando directamente el terminal en el diccionario: `if id_terminal_compra in terminal_a_socio`. Esto reduce el tiempo de búsqueda a $O(1)$.
- **Acumulación Dinámica:** Durante la lectura de las transacciones, el algoritmo inicializa y actualiza los contadores de forma dinámica. Si un cliente es nuevo para un socio, lo registra; si ya existe, incrementa su valor: `conteo_por_socio[socio_actual][id_cliente] += 1`.
- **Filtro de Inactividad:** Previo a calcular máximos, el sistema verifica el estado del diccionario de cada socio. Si un socio no registró clientes: `if not clientes_del_socio`, el flujo se desvía automáticamente para imprimir el formato de error controlado correspondiente, devolviendo el valor -1.
- **Evaluación y Desempate Simultáneo (lambda):** Para encontrar al cliente más fiel, se utiliza la función integrada `max()`, apoyada en esta función `(lambda x: (compras[x], -x))` Esta evaluación combinada permite priorizar el valor máximo de compras y, como segundo criterio simultáneo, escoge el menor ID numérico posible, resolviendo los empates.

2.5. Salida del programa

```
***  3 3 5
     1 10
     1 20
     2 30
     100 10
     100 20
     200 10
     200 20
     300 30
     1 100
     2 300
     3 -1
```

Figura 2: Salida del programa